Haoyang Tang

tanghy@idsse.ac.cn | Sanya, Hainan, China | github.com/wojiaonioo | wojiaonioo.github.io

M.S. student in physical oceanography | Differentiable ocean models, GPU acceleration, hybrid modeling and data assimilation

Education

Institute of Deep-sea Science and Engineering, Chinese Academy of Sciences **Physical Oceanography, M.S.** 2025.09 - present

Lanzhou University **Atmospheric Sciences, B.S.** 2019.09 - 2023.06

Project

TorchOcean: a differentiable GPU ocean model (sole developer) 2026.03 - present

Rewrote the numerical algorithms of MITgcm from scratch in PyTorch, so the whole model is differentiable; about 120,000 lines of Python and 14,000 lines of CUDA C++, more than 3,500 commits.

Model development and validation

- Regional, global latitude–longitude and cubed-sphere grids; hydrostatic and nonhydrostatic equations; open boundaries, KPP vertical mixing and tidal forcing.
- Instrumented the MITgcm source to compare every stage of a time step: all 6 official test cases pass, and 5 of them stay bit-for-bit identical after 600 steps.
- Compared the gradient of a 50-day global simulation point by point with the MITgcm adjoint: the largest normalized difference over 238,000 control variables is 5.5×10^{-11}.
- Built a verification matrix that checks results, gradients, speed and memory for each case and GPU count, replacing the former 7,450 unit tests.
- Reported two bugs to MITgcm: one is fixed (issue #989), and the fix for the other has been approved by the maintainers (issue #1039).

GPU acceleration

- On a 960×960×90 grid, one step on two A800 GPUs went from 786 ms to 161 ms, as fast as MITgcm on 576 CPU cores; one GPU takes 3.3 ns per grid cell per step in double precision, the same order as the GPU ocean model Oceananigans.
- Restructured the time-step code to remove torch.compile graph fragmentation and per-step recompilation: a step used to break into 131 graphs with 6 functions running uncompiled; now it breaks only at the solver and does not recompile in steady state.
- Wrote 43 CUDA kernels that each fuse the multi-step computation of KPP mixing, implicit vertical advection, tracer advection or horizontal viscosity; the KPP boundary-layer-depth kernel cuts kernel launches from 2,786 to 2 and is more than 80 times faster than the PyTorch version; 6 kernels have hand-written backward passes.
- Fixed a deadlock caused by the compiler reordering communication; two-GPU parallel efficiency is 85% to 93%.
- Checkpoints are kept in GPU or host memory depending on free GPU memory, and the solvers use implicit differentiation: a 100-step gradient on a 480×456×90 grid on one GPU went from 421 s to 30 s, about 4 times the simulation time.

Hybrid modeling and data assimilation

- Replaced the bottom-drag formula with a 337-parameter neural network; online training recovered the formula used to generate the truth, and the test loss fell to 1/498 of its value before training.
- Replaced KPP vertical mixing with a neural network trained only on temperature and salinity observations: test-period temperature error fell by 80% to 88%; analysed which mixing parameters such observations can constrain.
- 4D-Var with automatic differentiation in place of a hand-written adjoint: under the same setup the cost function falls to about 1/11.7 of its initial value, as with the MITgcm adjoint; in a regional twin experiment, assimilation reduced temperature error by 81%.
- Inverted the bottom-drag coefficient in a South China Sea experiment, reducing its error from 20% to 5.1%; estimated a surface heat-flux correction from temperature profiles measured at Ocean Station Papa.
- Next: couple the WRF atmospheric model into a differentiable ocean–atmosphere model and develop AI-hybrid 4D-Var on it.

Skills

Programming: PyTorch (torch.compile, custom operators, autograd), CUDA C++, Python, Fortran, NCCL / MPI
Scientific computing: ocean numerical models, automatic differentiation and adjoints, variational data assimilation
Tools: Git, SLURM, Nsight, Claude Code, Codex